EduVulcan2.0

Opis techniczny projektu — uzasadnienie decyzji, architektura, kontenery, baza danych i uruchamianie

Cel: internetowy dziennik demonstracyjny, w którym administrator zarządza kontami, nauczyciel pracuje z ocenami i frekwencją w swoich przydziałach, a uczeń przegląda własne dane. System zawiera również pocztę szkolną. Moduł planów lekcji jest przygotowany w kodzie, ale wymaga migracji na docelowym Linuxie.

1. Dlaczego aplikacja internetowa?

Jedna aplikacja działa na serwerze, a użytkownik korzysta z niej przez przeglądarkę. Nie trzeba instalować osobnego programu na komputerach uczniów i nauczycieli. Zmiany w logice wprowadzamy centralnie, na serwerze.

2. Dlaczego Python i Flask?

Python 3.13: czytelna składnia, szybki rozwój prototypu i bogaty ekosystem. Flask: lekki framework, który pozwolił dobrać tylko potrzebne biblioteki i zrozumieć przepływ żądań HTTP. Django oferuje więcej gotowych mechanizmów, lecz na potrzeby szkolnego prototypu Flask dał prostszą, bardziej kontrolowaną strukturę.

3. Dlaczego MySQL, SQLAlchemy i Jinja2?

MySQL 8.4 to relacyjna baza danych: oceny są powiązane z uczniami i przedmiotami, a przydziały z nauczycielami i klasami. SQLAlchemy pozwala opisać tabele jako klasy Pythona, co ułatwia zarządzanie relacjami i zapytaniami. Jinja2 tworzy strony HTML po stronie serwera — nie trzeba pisać osobnego frontendu SPA, aby pokazać dane z bazy.

4. Jakie kontenery Docker stworzyliśmy?

WEB — eduvulcan2-web-1: obraz budowany z Dockerfile na bazie python:3.13-slim. Instaluje pakiety z requirements.txt, zawiera kod Flaska, HTML i CSS, uruchamia python app.py i słucha na porcie 5000 wewnątrz kontenera.

DB — eduvulcan2-db-1: gotowy obraz mysql:8.4. Przechowuje dane aplikacji; ma healthcheck i własny wolumen danych.

Dlaczego dwa? Oddzielenie odpowiedzialności. Można zastąpić kontener WWW bez usuwania danych. Aplikacja i baza mogą być diagnozowane osobno. Docker Compose definiuje obie usługi, ich konfigurację, zależności i sieć.

5. Jak działa komunikacja i mapowanie portów?

Przeglądarka → serwer Linux:10114 → kontener WEB:5000 → sieć Compose → db:3306. Zapis 10114:5000 oznacza port hosta i port kontenera. MySQL nie ma mapowania 3306 na publiczny port hosta — aplikacja łączy się przez wewnętrzną nazwę usługi db.

6. Dlaczego Dockerfile i Docker Compose to nie to samo?

Dockerfile jest instrukcją budowania obrazu pojedynczego środowiska (naszej aplikacji Python). Docker Compose opisuje wspólne działanie kilku usług, ich porty, zmienne środowiskowe, healthcheck oraz wolumeny. Obraz to przepisane środowisko, kontener to jego uruchomiona instancja.

7. Dlaczego wolumen i backup?

MySQL zapisuje dane w wolumenie mysql_data, który nie znika przy zwykłym odtwarzaniu kontenera. Kopie mysqldump chronią dodatkowo przed błędami i pozwalają odtworzyć bazę. Wolumen nie zastępuje kopii zapasowej. Polecenia docker compose down -v nie wolno używać na tym serwerze, bo usuwa wolumen projektu.

8. Jakie mamy tabele?

Podstawowe: users, students, subjects, grades, teacher_assignments. Rozszerzenia: grade_metadata (typ i opis oceny), attendance (obecność), mail_messages, mail_recipients i mail_attachments (wiadomości), class_timetables (plany PDF po migracji). Klucze obce utrzymują powiązania danych. Kolory ocen informują o rodzaju; wszystkie oceny mają jednakową wagę.

9. Dlaczego kontrola ról jest po stronie serwera?

Ukrycie przycisku w HTML nie jest zabezpieczeniem. Flask sprawdza, kto jest zalogowany i jakie ma uprawnienia. Nauczyciel może edytować dane tylko dla przypisanej pary klasa–przedmiot, a uczeń widzi własne oceny i frekwencję. Niezgodne żądania mogą otrzymać odpowiedź HTTP 403.

10. Jakie mechanizmy bezpieczeństwa zastosowaliśmy?

Flask-Login zarządza sesjami. Hasła są przechowywane jako hashe, nie w jawnym tekście. Flask-WTF CSRFProtect chroni formularze. Sekrety pobieramy z .env zamiast umieszczać je w kodzie. Załączniki mają ograniczenia formatów i rozmiaru oraz kontrolę uprawnień. To zabezpieczenia prototypu, a nie profesjonalny audyt produkcyjny.

11. Dlaczego GitHub, gałęzie i GitHub Actions?

Git umożliwia śledzenie historii projektu. Gałęzie feature i Pull Requesty rozdzielały prace nad funkcjami od działającego main. GitHub Actions sprawdza składnię i uruchamia testy pytest z tymczasową bazą SQLite, bez dotykania szkolnego MySQL. Testy w stanie Queued nie są jeszcze zaliczone.

12. Jak uruchomić projekt?

Nowa instalacja: sklonować repozytorium, przygotować .env (MYSQL_PASSWORD, MYSQL_ROOT_PASSWORD i SECRET_KEY o długości co najmniej 32 znaków), wykonać docker compose up -d --build, utworzyć schemat db.create_all() w nowej bazie i interaktywnie założyć admina przez create_admin.py. Aktualizacja istniejącej instalacji: najpierw backup, potem git pull, docker compose build web, odpowiednie skrypty init_*.py, a na koniec docker compose up -d web. Konkretne komendy i diagnoza błędów są w osobnym README.

13. O czym pamiętać podczas prezentacji?

Najważniejsza decyzja to rozdzielenie aplikacji WWW i bazy danych. Opowiedz o portach, relacjach tabel, rolach, wolumenie i testach. Nie twierdź, że projekt jest w pełni produkcyjny ani że plany PDF działają na serwerze bez sprawdzenia migracji.

Przykładowe pytania nauczyciela

Dlaczego nie jedna baza w pliku? MySQL daje osobny serwer relacyjnej bazy, obsługę połączeń i przejrzyste relacje między danymi.

Co oznacza 10114:5000? Port 10114 serwera Linux przekierowuje ruch do portu 5000 aplikacji w kontenerze.

Co robi healthcheck? Sprawdza stan serwera MySQL; web zależy od gotowej bazy.

Dlaczego nie można ufać formularzowi HTML? Użytkownik może zmodyfikować żądanie; walidacja i uprawnienia muszą być sprawdzane przez backend.

Czy dane zostaną po przebudowie web? Tak, znajdują się w MySQL i jego wolumenie, jeśli wolumen zostanie zachowany.